

Native compressed braid certificates: Polynomial-size three-braid recognition and singleton forests toward quasi-polynomial unknot recognition

Research continuation prepared for the ProveIt project

8 October 2026

Abstract

We develop a certificate-producing unknot recognizer whose input is a binary straight-line program for a braid word, rather than an expanded crossing list. For every three-strand input it decides whether the closure is a knot and, when it is, whether it is trivial, in deterministic time polynomial in the grammar's bit size. The algorithm combines the existing three-braid criterion in ProveIt with explicit compressed reduction in $C_2 * C_3$. Its central operation is one longest-common-prefix query followed by at most one syllable merge. For s input rules and B bits of represented-length metadata, a direct implementation allocates $O(s^2(B+1))$ string rules. An independently structured verifier checks polynomial-size certificates without repeating longest-prefix searches.

We lift singleton-generator connected-sum decomposition to native compressed inputs of arbitrary strand count. Every three-strand factor is removed from the difficult part of the problem without expansion. If κ_i is the minority-sign count of an unresolved wider factor, a complete exponential fallback gives

$$\text{poly}(g) + \sum_{i \in \text{exc}} 2^{O(\kappa_i)},$$

where g is input bit size. Thus only the largest *exceptional* minority count controls the exponential term. This is a restricted-input quasi-polynomial result, not a general quasi-polynomial algorithm for arbitrary knot diagrams.

The package supplies source, an independent replay layer, 42 passing unit tests, an exhaustive 87,381-word comparison, compressed-capacity checks, and paired timings. It also proves an exponential bit-size obstruction to materializing Burau matrices on compact conjugate inputs. Polynomial compressed conjugacy in virtually free groups is established prior theory; our contribution is the explicit certificate-oriented specialization, native compressed decomposition, and reproducible implementation, not a claim of priority for that general complexity fact.

Deliverable status. Mathematical proofs and executable research implementation; no proof-assistant certification. Actual full-checkout integration and whole-pipeline timings have not been run. No remote files were changed.

Inspected snapshot: 47002f64b9a97f64edc8e8b8f793b83983b719f9

Contents

1	Research objective and the precise increment	3
1.1	Why change the input representation?	3
1.2	Relation to the inspected project	3
1.3	Prior mathematical complexity results	3
1.4	What remains outside the result	4
2	Input model, finite summaries, and bit-size conventions	4
3	The three-braid criterion, with an algebraic audit	5
4	Compressed free-product multiplication and cyclic reduction	6
4.1	The exact string interface	6
4.2	Discovery pseudocode	8
5	Polynomial grammar growth and bit complexity	8
6	Proof-carrying recognition and independent replay	9
6.1	Certificate contents	9
6.2	Checking a product without searching for its cancellation	10
6.3	Resource semantics	11
7	Why full Burau matrices can defeat compression	11
8	Singleton forests directly on compressed braids	12
8.1	The topological split and its provenance	12
8.2	Counting without enumerating occurrences	12
8.3	Why closure equivalence is not a substring rewrite	13
9	The three-braid-exempt minority bound	14
9.1	Strict scope gain over counting all factors	15
9.2	The precise missing general theorem	15
10	Executable validation and measured performance	15
10.1	What was actually run	15
10.2	Paired benchmark design	16
10.3	Capacity, grammar sensitivity, and negative controls	17
11	Integration contract and proposed project corrections	18
11.1	Additive integration rather than an assumed patch	18
11.2	Corrections to avoid in complexity claims	18
11.3	Operational review before promotion	18
12	Further research questions and topics	19
13	Conclusion	20
A	Reproduction and artifact map	21
B	Claim ledger	21

1 Research objective and the precise increment

1.1 Why change the input representation?

The repository already has a linear-time recognizer for explicit three-braid knot closures. It also has substantially more general exact recognition machinery and increasingly capable compressed free-group operations [1]. Reimplementing the explicit three-braid criterion would not advance that work. The missing interface addressed here is different: a braid may be presented by a small acyclic grammar while its literal Artin word is exponentially longer. Expanding that grammar before using an otherwise efficient recognizer destroys the available compression.

We therefore ask for a decision algorithm measured in the number of bits actually supplied. Its output must not include the full expanded cyclic normal form, an enormous integer matrix, or a sequence of individually expanded conjugations. An affirmative answer must still be tied to the precise input braid. A matching numerical fingerprint, a small-looking intermediate presentation, or an exhausted computation is not a certificate.

This work targets three related bottlenecks. First, make three-braid closure recognition native to grammar inputs. Second, make every algebraic transformation independently replayable in that representation. Third, preserve compressed easy factors when an arbitrary-strand braid is split into connected-sum factors. The third point is important: expanding a huge easy factor simply because another factor needs a general backend would forfeit the main gain.

1.2 Relation to the inspected project

The maintained file `fast/fastunknot/braid.py` already uses the quotient $B_3/Z(B_3)$ and exponent sum to decide explicit three-braid closures. Its alternate matrix backend uses Burau at $t = -1$. The string kernel in `compressed_words.py` implements exact SLP equality and compressed free-group operations. Report 19 studies explicit rank-two substring replacements. Report 34 studies explicit singleton splitting, minority-sign bounds, and endpoint descent [1, 2, 3]. We use those results as starting points, not as newly proved project achievements.

The supplied `strings.py` is an attributed, source-derived adaptation of the maintained exact string kernel. It is neither a byte-identical checkout nor the entire original module. New producer and replay modules implement free-product, not free-group, reduction. In particular, a yy boundary becomes y^2 ; it is not treated as a free-group cancellation. The existing general group-search SLPs are not automatically braid-word SLPs, so they cannot be passed to this backend merely because their storage formats look similar.

1.3 Prior mathematical complexity results

Exact equality of SLP strings is deterministic polynomial time; we use the standard primitive in the form developed through Plandowski’s method and explained by Schleimer [4]. Holt, Lohrey, and Schleimer prove polynomial-time compressed word and simultaneous conjugacy algorithms for fixed hyperbolic groups [5]. Since $C_2 * C_3$ is virtually free, those results already imply a polynomial algorithm for the relevant quotient conjugacy tests. Keeping exponent sum separately handles the central lift. We make no novelty claim for this existence consequence.

The purpose of our direct construction is a transparent implementation contract: explicit syllable reduction, concrete size accounting, short replay witnesses, and compatibility with the repository’s exact string interface. Its cost analysis exposes the dependence on the equality kernel rather than hiding it behind a claimed small exponent.

1.4 What remains outside the result

The general target is still an end-to-end bound for arbitrary knot diagrams. Lackenby’s July 2026 preprint gives a new hierarchy algorithm; its Section 9 explicitly distinguishes iteration counting from costs of operations, and Section 10 bounds refined hierarchy length in terms of initial complexity [10]. Neither those statements nor the present algebraic module supplies a complete implementation of the missing accelerated geometric operations.

Input compression also differs from compressed *homology output*. Kelomäki and Schütz analyze exponential behavior of explicit Khovanov constructions and develop compressed descriptions in restricted settings [11]. Our backend never builds a Khovanov complex for a solved three-braid. It does not claim that a polynomial-size description of homology can always be discovered from an arbitrary compact input.

2 Input model, finite summaries, and bit-size conventions

Definition 2.1 (Binary braid grammar). For a specified strand count m , a grammar consists of a distinguished empty rule $X_0 = \varepsilon$, followed by s rules

$$X_i = \sigma_j^\epsilon \quad (1 \leq j < m, \epsilon \in \{-1, 1\}), \quad \text{or} \quad X_i = X_a X_b \quad (a, b < i),$$

and a root index r . Its value is an Artin word, and its topological meaning is the ordinary closure of that word on m strands. The grammar has no power, inverse, recursive, or probabilistic rule type.

Powers and inverses are conveniences of the builder, not extra input semantics. Binary powering creates $O(\log(q + 1))$ concatenation rules for a positive power q . Inversion reverses concatenations and negates generator signs with memoization. Arbitrary shared DAGs are allowed; a small number of signed runs is not required.

Write g for the total input encoding size in bits, including indices and the strand count. Let N_i be the expanded length of rule i , and set

$$N = N_r, \quad N_* = \max_{0 \leq i \leq s} N_i, \quad B = \lceil \log_2(N_* + 2) \rceil.$$

We use N_* rather than N because the public validator processes all supplied rules, including unreachable ones. A root of length two can coexist with a large unused rule. For binary acyclic grammars $N_* \leq 2^s$, so $B = O(s + 1)$ and $s \leq g$ up to a fixed encoding convention. There is no hidden assumption that arithmetic on an N -bit matrix entry costs one operation.

Lemma 2.2 (Finite metadata). *For fixed m , lengths, exponent sums, and strand permutations are computable in polynomial bit time in g . For $m = 3$, these computations require $O(s)$ additions or constant-size permutation compositions on integers of $O(B)$ bits.*

Proof. A terminal has length one, exponent ϵ , and the adjacent transposition $j \leftrightarrow j + 1$. Concatenation adds the two lengths and exponents and composes the two permutations. Strictly earlier references permit one topological pass. Absolute exponent is bounded by expanded length. The cycles of the root permutation are the components of the closed braid, so a three-cycle is exactly the one-component condition when $m = 3$. $\square$

The implementation rejects malformed rules, out-of-range generators, forward references, non-integer indices, and Boolean values masquerading as integers. It distinguishes a multi-component closure, reported as `LINK`, from a nontrivial knot, reported as `KNOTTED`. The knot-detection theorems

apply only after the component check. For $m = 1$ the empty braid closes to an unknot, whereas the empty three-braid closes to a three-component link.

The native JSON schema is deliberately small:

```
{
  "strands": 3,
  "rules": [
    ["e"],
    ["g", 1],
    ["g", 2],
    ["c", 1, 2]
  ],
  "root": 3
}
```

This is the braid $\sigma_1\sigma_2$, not a planar-diagram record. Binding it to a separately supplied diagram requires an additional certified conversion; no such binding is silently assumed.

3 The three-braid criterion, with an algebraic audit

This section records the criterion already used in the project and explains why the compressed backend may use it. It also separates a necessary invariant from a complete classification in this special domain.

Put $a = \sigma_1$, $b = \sigma_2$, $x = aba$, and $y = ab$. The standard presentation is

$$B_3 = \langle x, y \mid x^2 = y^3 \rangle, \quad c = x^2 = y^3, \quad G = B_3 / \langle c \rangle = C_2 * C_3.$$

Exponent sum e sends a, b to 1, so $e(c) = 6$. Consequently the map

$$B_3 \longrightarrow G \times \mathbb{Z}, \quad \beta \longmapsto (\bar{\beta}, e(\beta))$$

is injective: an element in its kernel is c^q with $6q = 0$. The same observation handles conjugacy lifts once a quotient conjugator is known.

Write $z = y^2$. The quotient syllables are x, y, z , with

$$x^{-1} = x, \quad y^{-1} = z, \quad z^{-1} = y, \quad a \mapsto zx, \quad a^{-1} \mapsto xy, \quad b \mapsto xz, \quad b^{-1} \mapsto yx.$$

A reduced word alternates between the x factor and the $\{y, z\}$ factor. A cyclically reduced word of length greater than one has its first and last syllables in different factors.

Theorem 3.1 (Existing three-braid closure criterion). *Let $\beta \in B_3$ have a one-component closure, exponent e , and cyclically reduced quotient word C . Its closure is the unknot exactly in the following cases:*

e	C up to cyclic rotation
2	y
-2	z
0	a length-four word with two x 's, one y and one z .

In particular, the decision needs the exact exponent and at most four syllables of the cyclic core, not the full core when it is longer.

Proof. Bennequin's inequality and the inequality for the mirror imply $|e| \leq 2$ for a three-braid unknot [6]. At $t = -1$, the reduced Burau matrices are

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad B_0 = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}.$$

The braid Alexander formula specializes, for three strands, to

$$\det K = |\det(I - M)| = |2 - \operatorname{tr} M|, \quad M = \rho_{-1}(\beta),$$

where the usual Laurent-unit ambiguity disappears after absolute value [7]. An unknot therefore has $\text{tr } M \in \{1, 3\}$.

We audit the corresponding small-trace quotient classes. A nontrivial cyclically reduced word of length greater than one can be rotated to $xy^{\epsilon_1} \cdots xy^{\epsilon_q}$ with $\epsilon_i \in \{1, 2\}$. Up to overall sign and conjugation by $\text{diag}(1, -1)$, its matrix is a product of

$$L = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad R = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Indeed $\rho(xy) = -L^{-1}$ and $\rho(xy^2) = -R^{-1}$. If the positive product contains p letters L and q' letters R , both positive, expand each factor as $I + E$ or $I + F$. All entries in all terms are nonnegative. Each of the pq' terms choosing precisely one E and one F has trace one, while the identity term has trace two. Hence

$$\text{tr}(\text{positive product}) \geq 2 + pq'.$$

Absolute trace three is possible only for one L and one R , and is attained there. A product with only one letter type has trace two. The remaining cyclic quotient cores are the empty word, x , y , and z ; their absolute traces are respectively 2, 0, 1, 1. Thus absolute trace one or three forces $C = y, z$, or the stated mixed length-four core.

The quotient also determines exponent modulo six. The two one-syllable cases have residues 2 and -2 ; the mixed length-four case has residue 0. With $|e| \leq 2$, the exponents are exactly those in the statement.

Conversely, the three cases are quotient-conjugate to the images of

$$ab, \quad b^{-1}a^{-1}, \quad ab^{-1},$$

respectively, with the same exponent. Lift a quotient conjugator to B_3 . The two resulting braids can differ only by c^q , and exponent equality forces $q = 0$. These three canonical closures are unknots by two elementary destabilizations. Conjugacy preserves braid closure, completing both implications. $\square$

For general strand count, determinant one is not an unknot certificate. Even here determinant one without the exponent restriction is insufficient. The implementation relies on the complete theorem in its stated domain, not on an unqualified polynomial-identity test.

4 Compressed free-product multiplication and cyclic reduction

4.1 The exact string interface

A string arena stores terminal syllables and binary concatenations as an interned DAG. For a node U , the arena supplies length, first and last syllables, concatenation, half-open slicing $U[i : j]$, and exact equality. Inversion in this section is the anti-morphism reversing the word and exchanging y, z while fixing x . It is *not* integer sign negation as in a free-group alphabet.

Let $E(S, B)$ be a deterministic polynomial bit bound for exact equality on two strings in an arena with S rules and B -bit length metadata. The standard SLP equality theorem gives such an E [4]. We leave its exponent symbolic. Binary search using exact substring equality computes a longest common prefix in at most $B + O(1)$ queries. A bounded direct DAG walk is a permissible fast path only because it falls back to this complete procedure.

Lemma 4.1 (One query, at most one merge). *Let U, V be reduced words in G . Set*

$$k = \text{LCP}(U^{-1}, V), \quad U' = U[0 : |U| - k], \quad V' = V[k : |V|].$$

Their reduced product is $U'V'$ unless both surviving boundary syllables belong to the C_3 factor and are equal. In that case replace the two boundary syllables yy by z , or zz by y . No further reduction is necessary.

Proof. The inverse-prefix equality cancels exactly a suffix of U against a prefix of V . If a remainder is empty, the other is already reduced. Otherwise maximality of k says that the two surviving boundary syllables are not inverses. Distinct factors therefore require no action. Two x syllables would be inverses, as would y, z in either order; these cases contradict maximality. The only remaining same-factor pairs are yy and zz , with the products stated. On either side of the new C_3 syllable, any existing neighbor is an x , because the original words were reduced. Thus no new same-factor adjacency is created. $\square$

The algorithm processes input rules in order. A terminal is replaced by its two-syllable image. A concatenation invokes Lemma 4.1 on the already reduced child roots. Both children may share large subgraphs; no rule requires enumeration of occurrences in the expanded value.

Lemma 4.2 (A compressed cyclic core and conjugator). *Let W be reduced. Define*

$$t = \text{LCP}(W, W^{-1}, \lfloor |W|/2 \rfloor), \quad P = W[0 : t], \quad R = W[t : |W| - t].$$

If $|R| \leq 1$ or the endpoints of R have different factors, put $C = R$, $Q = P$. Otherwise $R = fMf$ for $f \in \{y, z\}$, and put

$$C = Mf^2, \quad Q = Pf.$$

Then C is cyclically reduced and $W = QCQ^{-1}$ in G .

Proof. The prefix equality gives $W = PRP^{-1}$. A longer inverse pair at the endpoints of R would contradict maximality of t . If the endpoints have the same factor and there is more than one syllable, they must therefore be equal C_3 syllables. The interior M starts and ends with x whenever nonempty; since R was reduced, this same-factor endpoint case has length at least three. The word Mf^2 is consequently cyclically reduced. Finally,

$$(Pf)(Mf^2)(Pf)^{-1} = PfMf^2f^{-1}P^{-1} = PfMfP^{-1} = W.$$

All equalities use $f^3 = 1$. The other cases are immediate. $\square$

4.2 Discovery pseudocode

```

validate the binary grammar and compute length, exponent, permutation
if the permutation is not a three-cycle: return LINK
if abs(exponent) > 2: return KNOTTED with the finite witness
normal[0] = empty
for rule i in topological order:
    if terminal: normal[i] = the two-syllable generator image
    else:
        k = LCP(inverse(normal[left]), normal[right])
        normal[i] = trim k from the adjoining ends; merge once if needed
        record k
(core, conjugator, trim, merge_flag) = cyclic_reduce(normal[root])
read core only if its exact length is at most four
apply the exponent/core criterion and emit the reduction certificate
independently replay the certificate before publishing a decision

```

The procedure finds the core; it is not a certificate-existence argument. Its number of rule reductions is exactly bounded by the input DAG, unlike unrestricted searches through Whitehead or Markov moves.

5 Polynomial grammar growth and bit complexity

A polynomial-time primitive is not enough if the calling program creates exponentially many intermediate rules. This section accounts for that growth explicitly.

Lemma 5.1 (Slice and height bounds). *An interval of a binary string grammar of height h can be represented using $O(h)$ new concatenation nodes without increasing height. The reduced representative of an input rule at dependency depth d can be maintained with height at most $2d + 2$.*

Proof. Only the two boundary paths of an interval can be partially retained. Every completely included subtree is reused. Reassembling the retained pieces in the original tree shape needs at most a constant number of nodes per boundary-path level; deleting empty pieces does not increase height. Reversal with syllable inversion preserves height. A terminal image has height two. In a product reduction, slicing does not raise child height, and reassembly uses at most two additional concatenations. Induction on input dependency depth proves the second bound. $\square$

Theorem 5.2 (Native compressed three-braid recognition). *For a binary SLP describing a three-braid, a complete component-and-unknot decision and a replay certificate can be produced in deterministic polynomial bit time in g . More explicitly, there is an implementation with*

$$S = O(s^2(B + 1)) = O(s^3 + s^2)$$

allocated string rules and at most $O(s(B + 1))$ exact string-equality queries. Its discovery time is bounded by

$$O\left(s(B + 1) E(Cs^2(B + 1), B + O(1))\right) + \text{poly}(s, B) \quad (1)$$

for an absolute constant C . Here E is any fixed deterministic polynomial SLP-equality bound.

Proof. There are at most s product reductions and one cyclic reduction. Each has one prefix query, with at most $B + O(1)$ bisection steps: all syllable lengths are at most twice the corresponding input lengths, and all scratch substrings have length at most $2N_*$. Each bisection step needs a constant

number of slices; Lemma 5.1 bounds their new nodes by $O(s)$. The final trimming and reassembly for each product also adds $O(s)$ nodes. Memoized inversion adds at most a constant multiple of the number of reachable nodes it has not yet inverted. Thus total allocation is $O(s^2(B + 1))$.

The equality method operates on this bounded arena. Metadata arithmetic has $B + O(1)$ bits. Topological validation, finite summaries, dictionary operations under conservative polynomial worst-case accounting, and the other DAG traversals contribute a polynomial term. Equation (1) follows. Since $B = O(s + 1)$, it is polynomial in the input bit size. Correctness follows from Lemmas 4.1 and 4.2, induction over rules, and Theorem 3.1. $\square$

The theorem does not advertise a cubic-time implementation. The cubic expression bounds *allocated string rules*, not the running time of equality. Nor does an abstract work counter establish a CPU-instruction bound. Completeness concerns the uncapped procedure; the research CLI uses finite allowances and can return inconclusive. Increasing allowances by a doubling schedule until discovery and replay finish recovers the complete procedure with polynomial overhead. It is nevertheless a genuine polynomial algorithm in compressed input size: both call count and representation growth have been bounded before invoking the polynomial primitive.

Corollary 5.3 (Compressed conjugate sleeves). *Let U be any braid SLP of size h , and let γ be one of the three canonical unknots in Theorem 3.1. A grammar for $U\gamma U^{-1}$ has size $O(h)$, and its closure is decided without expanding U . The same conclusion holds with any fixed three-braid γ , including knotted examples.*

Proof. Memoized inversion and concatenation add $O(h)$ rules. Apply Theorem 5.2; no conjugating decomposition has to be supplied separately to the recognizer. $\square$

This class includes grammars with nested reuse and many signed runs. It is strictly more general as a representation class than a word with logarithmically many explicit twist blocks. Conversely, an arbitrary explicit word need not have a sublinear grammar. Constructing an identity-sized grammar from it preserves polynomial correctness but is generally slower than the maintained linear explicit gateway.

6 Proof-carrying recognition and independent replay

6.1 Certificate contents

A versioned three-braid certificate contains the original grammar, the recomputed finite summaries, a string-rule table, the reduced root for every input rule, and one cancellation length for every concatenation. It also gives the final cyclic trim, the possible endpoint-merge flag, a core root, and a conjugator root. Exact long integers such as lengths and exponents use canonical signed hexadecimal strings, avoiding a decimal-conversion limit when lengths have thousands of bits. Node indices remain ordinary bounded integers.

The certificate binds to the supplied input by exact canonical JSON comparison, not by trusting a digest. A SHA-256 digest is retained as an integrity aid but is not a mathematical equality oracle. In particular, the verifier rejects a Boolean substituted for an integer generator even though a programming language might otherwise compare those values as equal.

The arena can contain scratch strings. The verifier therefore does not infer that every arena node is a reduced group word. It computes a reducedness flag compositionally and requires the claimed normal roots, cyclic core, and conjugator to have that property. References must point to earlier well-formed nodes. The verifier imports all strings into a fresh arena and recomputes lengths and endpoints; producer metadata is not authoritative.

6.2 Checking a product without searching for its cancellation

For an input concatenation, let U, V, W be the independently reconstructed child and claimed result roots, and let k be the claimed cancellation length. The verifier checks

$$0 \leq k \leq \min(|U|, |V|), \quad (U[|U| - k : |U|])^{-1} = V[0 : k]$$

by exact string equality. It then examines the surviving endpoints. A residual inverse pair is forbidden. Equal C_3 endpoints must be merged, while different factors must be concatenated without a merge. Finally, the resulting string is compared with W . The endpoint condition establishes maximality of k without recomputing an LCP.

For the cyclic witness the checker similarly verifies that the claimed prefix and suffix are inverse, checks that no further inverse trimming is possible, reconstructs the one permitted endpoint merge, and checks both C and Q . The classification is then recomputed from the source exponent and the certified core. The producer's final status is accepted only if it agrees.

Theorem 6.1 (Certificate soundness and size). *Every certificate accepted by the independent verifier gives the correct component or knot verdict for its supplied three-braid grammar. An honestly generated certificate has bit size*

$$O(g + S \log(S + 2) + sB), \quad S = O(s^2(B + 1)). \quad (2)$$

Its replay uses $O(s)$ exact string-equality queries and polynomial other work. Verification of an arbitrary supplied certificate is polynomial in the combined input and certificate sizes, subject to explicit resource caps.

Proof. The finite branch recomputes the root permutation and exponent, so its conclusions follow from component counting and Bennequin's obstruction. In the quotient branch, terminal roots are compared with the fixed generator images. Assume inductively that the child normal roots are correct. The checked inverse substring equality proves that the claimed cancellation is valid. The residual endpoint checks and reducedness prove maximality and validate the only possible merge, by Lemma 4.1. Exact equality with the claimed result completes the induction. The cyclic checks establish Lemma 4.2, after which the source exponent and Theorem 3.1 force the verdict.

Each arena rule has a constant number of references of $O(\log(S + 2))$ bits. Each input rule contributes a root index and a cancellation integer of $O(B)$ bits. The rest consists of the original input, finite summaries, and a constant number of roots and offsets. This proves (2). Each input product requires only a constant number of equality queries; so do each terminal and the final cyclic check. For valid producer output, the arena height is $O(s)$, and the checker's extra slices and inverse strings keep its allocation within the same polynomial envelope. For arbitrary supplied rule tables, their own heights and bit lengths replace those honest-output bounds, still giving polynomial verification in the combined encoding size. $\square$

The replay implementation is independently structured: it imports neither the producer's reduction object nor its prefix-search routine. Tests disable both the producer entry point and the arena's LCP method during replay. However, the two sides intentionally share the exact string kernel and the imported topology. This is a smaller and clearer trust boundary, not the elimination of every common dependency. No formal proof assistant has checked this implementation.

A valid affirmative witness can also be interpreted as a compact braid conjugator. Lift each quotient syllable by the fixed words $x = aba$, $y = ab$, $z = (ab)^2$. In the length-four case adjust by a constant-length cyclic rotation. The quotient conjugacy and exact exponent then lift to equality in B_3 . Thus the certificate does not conceal an exponentially long search for a conjugator; a polynomial-size grammar for one is available from its recorded data.

6.3 Resource semantics

A node, work, or external cooperative budget exception supplies no mathematical verdict. The CLI reports **INCONCLUSIVE** for its supported resource exception. Discovery and replay use fresh arenas, and a decision is published only after successful replay. The work counter charges abstract kernel operations, not every bit operation or elapsed second. Metadata validation and JSON parsing must also be covered by a host application's global size and time policy.

In the forest interface, arena caps apply separately to each child computation; they are not a global forest work allowance. A production integration should supply a shared deadline or work callback across all leaves and replay phases. An exception from an actual injected upstream kernel must be handled as that kernel's resource type, not mistaken for our local **Limit** class. The optional checkout check makes this boundary explicit.

7 Why full Burau matrices can defeat compression

The maintained matrix control is elegant and effective for explicit words. It is not automatically a polynomial-bit algorithm for SLP input merely because matrix powering uses logarithmically many multiplications.

Theorem 7.1 (Materialization obstruction). *There is a family of three-braid unknot grammars of size $O(\log(m+1))$ for which a full ordinary binary representation of the Burau matrix at $t = -1$ requires $\Omega(m)$ bits, although its trace is identically one. With $m = 2^k$, this is exponential in the grammar size.*

Proof. Let $u = ab^{-1}$ and $\beta_m = u^m(ab)u^{-m}$. Its closure is an unknot. Binary powering and shared inversion give the claimed grammar size. Write

$$Q = \rho(u) = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}, \quad T = \rho(ab) = \begin{pmatrix} 0 & 1 \\ -1 & 1 \end{pmatrix}.$$

The Fibonacci identity gives, for $m \geq 1$,

$$Q^m = \begin{pmatrix} a & b \\ b & d \end{pmatrix} = \begin{pmatrix} F_{2m+1} & F_{2m} \\ F_{2m} & F_{2m-1} \end{pmatrix}, \quad a = b + d.$$

The upper-right entry of $Q^m T Q^{-m}$ is

$$b^2 + a(a + b) = 3b^2 + 3bd + d^2.$$

Since $F_{2m} \geq 2^{m-1}$, that entry has at least $2m - 1$ bits. Conjugation nevertheless keeps the trace equal to $\text{tr } T = 1$. Hence any algorithm that materializes this full matrix in ordinary binary integer form has the stated output-size lower bound. $\square$

The obstruction is deliberately narrow. It does not rule out compressed arithmetic circuits, modular obstructions, symbolic cancellation of a recognized conjugator, or other matrix-informed algorithms. It rules out treating ordinary huge matrix entries as unit-cost scalars. Our quotient string representation avoids exactly that materialization obligation.

This example also distinguishes three measures that can otherwise be confused: the represented braid length, the size of a symbolic proof of its conjugacy, and the size of an integer matrix representation of the same braid. The first and third may be exponential while the second remains small. A complexity analysis must name the representation at every interface.

8 Singleton forests directly on compressed braids

8.1 The topological split and its provenance

For a braid word w on m strands, let $c_j(w)$ count occurrences of σ_j or σ_j^{-1} , without regard to sign. The singleton observation has classical antecedents in work of Van Cott [8]; the project already develops an explicit all-factor version [3]. We give the topological argument to specify exactly what is preserved by our compressed projections.

Lemma 8.1 (Singleton connected-sum split). *Suppose the closure of $w \in B_m$ is a knot and $c_j(w) = 1$. Delete that unique crossing and project the remaining letters onto indices less than j and greater than j , shifting the latter down by j . These give braids $w_L \in B_j$ and $w_R \in B_{m-j}$. Their closures are knots and*

$$\text{cl}(w) = \text{cl}(w_L) \# \text{cl}(w_R).$$

The conclusion is independent of the sign of the singleton crossing.

Proof. Cyclically rotate w to place the singleton crossing at the closure seam. Every remaining letter belongs to one of the two strand intervals, and letters from the two intervals commute: their generator indices differ by at least two. Thus the two subbraids can be drawn in disjoint side-by-side regions, with the sole crossing joining the distinguished closure arcs. A separating sphere through those two joining arcs exhibits their connected sum. The sign changes the local band convention but not the two summand knot types. If the two separate closures have r_L, r_R components, that joining merges one component from each, leaving $r_L + r_R - 1$. A one-component original therefore forces $r_L = r_R = 1$.

Projecting the original word instead of its cyclic rotation only cyclically rotates each projected subword. Their closures are unchanged. Thus the statement applies to the deterministic source-order projections used by the implementation. $\square$

Split at all indices with $c_j = 1$. Induction in Lemma 8.1 gives interval leaves and a connected-sum forest. A projection removes letters of other indices but neither changes nor duplicates occurrences of a retained index. Therefore every retained internal generator in a leaf is either absent or still has its original multiplicity. For a knot leaf none is absent, and, after all singleton cuts, each multiplicity is at least two.

8.2 Counting without enumerating occurrences

The same terminal rule may occur exponentially many times in the root value. Counting one occurrence per stored terminal would be wrong. Instead put multiplicity one on the root and propagate weights down the DAG in reverse topological order. At $X_i = X_a X_b$, add the weight of i to both a and b , including twice when $a = b$. At a terminal, add its accumulated weight to the unsigned generator count. This computes every c_j exactly with a linear number of propagation steps and polynomial-size integers.

There is a subtle input-validation issue when the strand count itself is binary encoded. An empty grammar with an enormous m must not cause allocation of an enormous permutation array. If any index $1 \leq j < m$ is missing, the braid permutation preserves both strand intervals separated by j ; its closure cannot be a knot. A missing index can be found by sorting the stored support and locating a gap. If no index is missing, then every one of the $m - 1$ labels appears in some terminal rule, and hence

$$m \leq s + 1. \tag{3}$$

Only then do we allocate and compose m -point permutations. This makes variable-strand component validation polynomial in actual input size, not in a potentially enormous unsupported strand count.

Lemma 8.2 (Native interval projections). *Given a strand interval $[\ell, h]$, its projected braid grammar can be computed using at most s nonempty rules, by deleting terminals outside the interval, shifting retained generator indices, and pruning empty concatenations. All singleton leaves together have polynomial total encoding size in g .*

Proof. Process each input rule once. A terminal maps either to a shifted terminal or to the distinguished empty node. A concatenation maps to the concatenation of its mapped children, with an empty child suppressed. Interning can only decrease the number of nonempty output rules. There are at most m leaves, and (3) applies to an input that passes the knot-component test. Thus at most $O(sm)$ rules are needed, with polynomial-size indices and generator labels. $\square$

Theorem 8.3 (Native compressed singleton-forest recognition). *There is a deterministic polynomial-bit procedure on arbitrary-strand binary braid grammars that computes and verifies their singleton decomposition and completely decides every input for which all leaves have at most three strands. This conclusion is independent of the leaves' expanded crossing counts. On other inputs it gives sound one-sided decisions and otherwise reports *INCONCLUSIVE*.*

Proof. Perform compressed counts and component validation, then construct all interval projections. One-strand leaves are unknots. A two-strand knot closure has odd exponent and is unknotted exactly when the exponent has absolute value one. Apply Theorem 5.2 to every three-strand leaf. The original knot is trivial exactly when all its connected-sum factors are trivial. For the nontrivial implication, intersect a spanning disk with a summing sphere, remove closed intersection curves by innermost-disk surgeries, and cut along the sole remaining arc; this gives spanning disks for the capped summands. Conversely, boundary-connecting summand disks gives a spanning disk for their connected sum. There are polynomially many leaves of polynomial grammar size, proving the time and certificate bounds. A wider leaf may be rejected by Bennequin's obstruction. Without such an obstruction or a separate complete backend it remains unresolved. No unresolved leaf is ever counted as an unknot. $\square$

The forest verifier recomputes source counts, the complete cut set, and every interval projection. It then invokes child verifiers and recomputes the connected-sum verdict. Projection code is shared between discovery and replay; the independently implemented component of replay is the three-braid algebra, not a second independent topology library. A tampered cut list or altered projected grammar is rejected.

8.3 Why closure equivalence is not a substring rewrite

Theorem 3.1 is a theorem about *closed* braids. It does not license replacing an arbitrary three-strand subword of a larger braid by any other word with the same closed knot. Such a substitution can alter how the tangle meets its environment. The singleton theorem supplies a genuine connected-sum separation, making the closed factor the correct object to decide. In other settings one must retain boundary and peripheral data, or prove equality in the braid group rather than equality of closed knot types. The implementation performs only the certified singleton separation, not unrestricted local closure replacements.

9 The three-braid-exempt minority bound

The compressed forest makes an existing minority-sign estimate more useful. One need not pay an exponential cost for the minority count of a factor that has already been decided in compressed form.

For each leaf i , let n_i be its expanded crossing count, m_i its strand count, e_i its exponent, and

$$\kappa_i = \min(n_i^+, n_i^-) = \frac{n_i - |e_i|}{2}.$$

These integers are computed directly on the leaf DAG. Let $\mathcal{E}$ be the set of leaves with more than three strands that pass the Bennequin test and are not otherwise decided by a certified easy backend. Define

$$\kappa_{\text{exc}} = \max_{i \in \mathcal{E}} \kappa_i, \quad \max \emptyset = 0.$$

For the delivered implementation, the easy complete backends are the at-most-three-strand ones; no additional wider-leaf backend is assumed in its actual verdicts.

Lemma 9.1 (Exceptional crossing bound). *Every unresolved leaf after all singleton cuts and its local Bennequin test satisfies*

$$n_i \leq 4\kappa_i, \quad m_i \leq 2\kappa_i + 1.$$

Proof. Each of the $m_i - 1$ internal generator indices occurs at least twice, so $m_i - 1 \leq n_i/2$. Passing the local test means $|e_i| = n_i - 2\kappa_i \leq m_i - 1$. Combining gives $n_i - 2\kappa_i \leq n_i/2$, hence $n_i \leq 4\kappa_i$, and then $m_i \leq n_i/2 + 1 \leq 2\kappa_i + 1$. $\square$

This elementary estimate is inherited from the explicit singleton/minority analysis [3]. What changes is the exceptional set: arbitrarily large compressed three-braid leaves have already been solved and are excluded.

Theorem 9.2 (Hybrid complete bound). *Attach any complete exact fallback that decides a k -crossing knot diagram in $2^{O(k)}$ bit time. On a binary braid grammar of bit size g , the singleton forest followed by compressed three-braid decisions and separate exceptional fallbacks is a complete recognizer with time*

$$\text{poly}(g) + \sum_{i \in \mathcal{E}} 2^{O(\kappa_i)} \leq \text{poly}(g) + |\mathcal{E}| 2^{O(\kappa_{\text{exc}})}. \quad (4)$$

It is polynomial in g when $\mathcal{E}$ is empty and quasi-polynomial in g whenever $\kappa_{\text{exc}} = O(\log^2(g+2))$.

Proof. All native compressed operations have polynomial cost by the previous theorems. Any knotted easy factor or local Bennequin obstruction terminates with a sound negative decision. Otherwise the original is an unknot precisely when all unresolved factors are unknots. By Lemma 9.1, an exceptional leaf expands to at most $4\kappa_i$ crossings on at most $2\kappa_i + 1$ strands. Expanding that leaf and constructing its ordinary diagram takes polynomial time in this size, absorbed by $2^{O(\kappa_i)}$. Apply the fallback independently to each leaf and sum the costs. The number of leaves is polynomial in g . Finally $2^{O(\log^2(g+2))} = (g+2)^{O(\log(g+2))}$. $\square$

A suitable theoretical fallback exists by building a reduced Khovanov complex over $\mathbb{F}_2$ and doing exact linear algebra. A k -crossing knot has 2^k resolutions and only $O(k)$ state circles, so the expanded chain dimension and all elementary construction and elimination costs are $2^{O(k)}$. Kronheimer–Mrowka’s rank-one detection theorem supplies the decision criterion [9]; the integral universal-coefficient comparison transfers rank-one detection to $\mathbb{F}_2$. The package does *not* include a

new full Khovanov implementation or claim to have wired this fallback into production. Equation (4) is the proved bound for that complete composition; the delivered forest module deliberately returns `INCONCLUSIVE` on unsupported leaves.

Corollary 9.3 (Conventional diagram-size formulation). *Suppose an n -crossing input is transformed, with verified closure-preserving provenance and polynomial cost and encoding size, into the grammar of Theorem 9.2. If its exceptional minority parameter is $O(\log^2(n + 2))$, the complete composed recognition algorithm takes $n^{O(\log n)}$ time.*

Proof. Substitute the polynomial grammar bound and the stated parameter bound into (4), and include transformation and verification costs. $\square$

9.1 Strict scope gain over counting all factors

Consider a singleton join of many conjugate three-braid sleeves. Each sleeve may have exponentially large minority count in its SLP size, yet all leaves are solved by Theorem 5.2. Thus $\kappa_{\text{exc}} = 0$ even while the largest minority count over *all* factors is exponential. Append one fixed wider factor through a singleton join. The exceptional parameter is now bounded by that fixed factor alone. The remaining decision cost does not depend exponentially on the sleeves' expanded lengths.

This is a genuine improvement in the sufficient parameter condition, not a claim that every knot has a cheaply discoverable singleton decomposition of this kind. Braids with no singleton generator can leave a single large exceptional factor. The supplied four-strand inconclusive control demonstrates the boundary of the native recognizer rather than pretending that a failed split is a certificate.

9.2 The precise missing general theorem

For a general quasi-polynomial result along this route one would need a constructive transformation, with controlled proof size and cost, that always exposes a polynomial-size compressed easy-factor forest and leaves $\kappa_{\text{exc}} = O(\log^2 n)$. We prove neither the existence nor the discoverability of such a transformation for arbitrary diagrams. Merely knowing that an unknot can be simplified, or that a certificate can be encoded compactly, would not bound a search for that certificate. Those are separate requirements, and the present backend removes only the cost of solving the easy leaves once they are available.

10 Executable validation and measured performance

10.1 What was actually run

All reported computations used CPython 3.13.5 on Linux x86-64. The package requires only the Python standard library for its algorithms and experiments. Tests, fixed random seeds, complete measured samples, operation counters, source hashes, and environment strings are retained in `results/`. The counts in this section are completed local checks, not counts inherited from an upstream README.

The local suite has 42 passing test methods. It includes malformed-DAG rejection, exact finite summaries, interval slicing, 300 random free-product products, 150 cyclic-core identities, Artin-relation contexts, conjugation and mirror invariance, forced polynomial equality fallback, resource exhaustion, and forged-certificate rejection. Expansion-forbidden tests replace the debugging

expansion method by a function that raises immediately. A replay-independence test disables both the producer entry point and LCP during verification.

The separate exhaustive run covers all

$$\sum_{j=0}^8 4^j = 87\,381$$

words in $a^{\pm 1}, b^{\pm 1}$ of length at most eight. Every word is converted to a grammar, recognized, and independently replayed, and its verdict is compared with a source-derived explicit stack control and a separately coded exact integer Burau control. The population contains 41,005 link closures, 22,208 unknot verdicts, and 24,168 knotted verdicts, with zero mismatches. Thus the number of one-component comparisons is 46,376, not 87,381. These are word instances with repetition of knot types, not distinct knots.

An additional 400 fixed-seed random shared grammars agree with both explicit controls after bounded expansion. Their quotient cores are also checked with direct probes disabled, forcing the exact equality fallback. The controls are independently coded algebraic checks, not a new homology oracle. In particular, this run does not claim a full Khovanov differential check or a full maintained **fastunknot** regression run.

10.2 Paired benchmark design

Both timed methods start from the *same native SLP input*. The baseline expands it and runs the source-derived explicit three-braid stack recognizer. The new method performs compressed discovery *and independent replay*. Grammar construction is outside both timings; validation and in-memory proof construction are inside the new path. JSON disk I/O is outside both. Seven measured rounds alternate AB/BA order after one untimed warmup of each method. All samples and the method definitions are retained in **benchmark.json** and **experiments/benchmark.py**.

The measured family is

$$\beta_k = (ab^{-1})^{2^k} (ab)(ab^{-1})^{-2^k}, \quad N_k = 4 \cdot 2^k + 2.$$

It is deliberately compressible and shares the grammar of its conjugator with its inverse. This is a test of a concrete representation bottleneck, not a hard-knot corpus. The table reports local medians in milliseconds; baseline/new is the ratio of those medians, not a theorem about wall-clock speed.

k	Crossings N_k	Input rules	New (ms)	Baseline (ms)	Ratio
0	6	9	0.186	0.016	0.08
4	66	17	0.297	0.058	0.20
8	1,026	25	0.438	0.586	1.34
10	4,098	29	0.492	2.191	4.45
12	16,386	33	0.547	8.625	15.76
14	65,538	37	0.757	35.281	46.61
16	262,146	41	0.857	142.376	166.18
18	1,048,578	45	0.880	599.129	680.73

Table 1: Paired native-input comparison on the positive sleeve family. New includes discovery and independent replay; baseline includes expansion and the source-derived stack control. Ratios below one favor the baseline.

Core type	k	Input rules	String rules	Produce (ms)	Replay (ms)
Unknot	64	137	141	0.861	1.170
Knotted	64	136	261	68.000	3.471
Unknot	256	521	525	3.084	4.357
Knotted	256	520	1029	1174.122	13.243
Unknot	1024	2057	2061	12.063	18.016
Unknot	4096	8201	8205	54.807	80.045

Table 2: Completed expansion-forbidden capacity checks. Times are single-run observations, not paired benchmark medians. The absence of a huge expanded baseline is not a speed measurement.

Strands	Leaves	k	Input rules	Verdict	Total (s)
12	4	16	173	Unknot	0.004
48	16	128	4285	Unknot	0.112
48	16	128	4284	Knotted	0.407
96	32	256	16765	Unknot	0.591

Table 3: Native singleton-forest capacity checks, including replay. Each leaf is a compressed three-braid sleeve; the negative case has one figure-eight core.

The small cases cost more under compressed discovery plus replay. This is why the explicit linear gateway should remain the default for ordinary explicit three-braid words. The large positive ratios demonstrate the benefit of avoiding expansion when the *input is already compressed*; they do not imply sublinear reading of an explicit million-crossing diagram. Nor are they whole-production-pipeline speedups.

10.3 Capacity, grammar sensitivity, and negative controls

For the largest completed positive capacity case, $k = 4096$, the represented word has $4 \cdot 2^{4096} + 2$ letters. The input uses 8,201 nonempty braid rules, the producer arena uses 8,205 string rules, and the compact JSON certificate has 349,628 bytes. Discovery and replay both complete without debug expansion. These are representation-capacity facts; no expanded baseline is attempted for this case.

The negative sleeve family replaces the middle word ab by $(ab^{-1})^2$, whose closure is a figure-eight knot. Although it is comparably compressible, its reduction presents a different prefix-comparison pattern. At $k = 256$ discovery is much slower than for the positive sleeve, while replay remains relatively inexpensive. This is useful evidence against a uniform small-constant performance claim. An exploratory larger mixed capacity sweep was externally stopped before a complete report was retained; the final capacity table restricts negative sleeves to the completed $k = 64, 256$ cases. No timeout is used as a measured speedup.

To test dependence on exact subtree identity, the reassocated family builds the same conjugator using

$$(ab^{-1})^m = a(b^{-1}a)^{m-1}b^{-1}$$

for the inverse side. With direct probes disabled, the $k = 4, 8, 16, 32$ cases all complete and replay successfully. The $k = 32$ producer records 1,550 equality splits, so this is not an identity-pointer-only success. It is still a structured family, not a substitute for adversarial grammar-shape testing.

The forest capacity run includes a 96-strand braid with 32 three-strand leaves at $k = 256$. Its input has 16,765 nonempty rules, despite a represented crossing count of $32(4 \cdot 2^{256} + 2) + 31$. The entire native recognition and replay succeeds. A 48-strand case with one knotted leaf returns

KNOTTED; an unsupported four-strand case with no singleton cuts returns **INCONCLUSIVE**. These controls exercise all-factor acceptance, one-factor rejection, and honest nondecision.

11 Integration contract and proposed project corrections

11.1 Additive integration rather than an assumed patch

The supplied implementation is additive. A suitable review location is a named directory such as `Topology/UnknotRecognition/research/compressed-braid-certificates/`. No numbered report slot is assumed. The first production promotion should be the input schema, producer, and verifier behind an opt-in native-SLP route. Existing explicit-word dispatch and legacy certificate readers should remain unchanged.

The three-braid producer and verifier accept an arena factory. The intended factory is the maintained `WordArena`; only its exact string interface is used, and the free-product anti-morphism is implemented separately. The package supplies `integration/upstream_check.py` to check that interface against an actual checkout and to compare actual imported explicit decisions. This check was *not run* in the delivery environment. Repository reading through the connected GitHub tool succeeded, but obtaining a full checkout in the execution container did not. The source-derived local kernel tests must not be relabeled as actual upstream integration.

11.2 Corrections to avoid in complexity claims

The following are precise qualifications, not allegations of newly discovered errors in the maintained source.

1. The existing explicit three-braid algorithm is already linear in its word length. The present polynomial bound is in *compressed input size*; these two statements concern different input models.
2. The local $O(s^2(B+1))$ estimate bounds grammar allocation, not total running time. Exact equality still contributes the polynomial $E(S, B)$.
3. A polynomial compressed word primitive does not bound the number of general search moves. Our three-braid producer has a bounded DAG pass; the maintained general group search needs a separate move-count and grammar-growth argument.
4. The inequality $n_i \leq 4\kappa_i$ is used only after all singleton cuts and a passed *local* Bennequin test. Applying it before those conditions is unjustified.
5. An unresolved factor is not discarded. The hybrid theorem requires a complete fallback; the delivered partial forest interface does not silently provide one.
6. Certificate replay shares the exact string kernel and imported knot theory. It is independent of producer search, not independently formalized mathematics.

11.3 Operational review before promotion

Production review should run the real factory-injection script, then the current full regression suite, preserving proof-format versioning and resource semantics. It should also check that a grammar-producing front end binds every compressed braid to the original diagram or accepted raw-braid input. A generated group presentation is not such a binding. For wider-leaf fallbacks,

retain separate source projections and check each backend’s certificate in the appropriate domain before combining statuses.

A service-facing verifier must limit input bytes, certificate bytes, declared rules, and integer sizes before substantial allocation. Internal abstract work caps should not be mistaken for hard wall-clock or memory limits. Cooperative external callbacks are useful, but a hosting process must enforce its own global policy. The supplied research CLI is not claimed to be a hardened adversarial-input service.

12 Further research questions and topics

The following questions are organized by the proof obligation they would remove. A successful answer should include both a mathematical size bound and a checked implementation, rather than only favorable examples.

Research question 12.1 (A sharper free-product string kernel). Can direct compressed reduction in $C_2 * C_3$ reduce the $O(s^2(B + 1))$ allocation envelope while improving worst-case equality cost? One concrete target is an $O(s^2)$ -size normal-form representation with deterministic prefix queries that avoid a separate binary search over B bits. The analysis must include all substrings and inverse nodes accumulated across the whole DAG.

Research question 12.2 (Representation-insensitive performance). How do equivalent balanced, skewed, reassociated, and deliberately nonshared input grammars change the cost of recognition? The positive sleeve benchmark favors exact sharing. A useful next corpus should preserve the represented braid while varying its parse independently, record equality splits and peak assertions, and identify a normalizing or balancing transformation whose own cost does not erase its benefit.

Research question 12.3 (Smaller proof objects). Can discovery discard equality scratch nodes and emit only the subgrammar needed for replay, with a proved near-quadratic or better certificate-size bound? Any compaction must preserve exact references and must not reintroduce exponential duplication. Compare producer-arena size, reachable proof size, serialized bytes, and fresh verifier allocation separately.

Research question 12.4 (An independently verified equality foundation). Can the finite-factor reduction and cyclic-conjugator lemmas be formalized, followed by the indexed split-and-compaction equality kernel? An intermediate milestone is a second structurally different equality backend used only for audit. Sharing the same kernel on producer and verifier is practical, but it remains the largest common algorithmic dependency.

Research question 12.5 (Compressed moves that reveal new singleton cuts). Can certified context-safe braid equalities, cyclic cancellation, and endpoint descent update occurrence multiplicities and interval projections without expansion? The goal is to expose singleton cuts hidden by a compact sequence of elementary moves. A result must bound the number of moves and aggregate grammar growth; repeated local improvement alone does not give such a bound.

Research question 12.6 (A broader exact family of easy leaves). Which wider braid families admit polynomial-in-SLP-size closure recognition with independently replayable certificates? Reducible or explicitly decomposed braid families are natural candidates, provided their boundary identifications are retained. Every additional exact easy-leaf family can be removed from $\mathcal{E}$ in Theorem 9.2, improving the exceptional parameter without changing its proof pattern.

Research question 12.7 (Four-strand input as a controlled next case). What additional information beyond exponent and a virtually free quotient is needed for a useful four-strand compressed backend? The concrete target is a certified subclass with an exact membership and decision procedure, not a claim that the three-braid constant-core criterion simply generalizes. The supplied unresolved four-strand example is a minimal interface test for this extension.

Research question 12.8 (Presentation-dependent minority parameters). How much can κ_{exc} decrease under a bounded, certified set of Markov and braid moves? This parameter is attached to a presentation and its certified decomposition, not claimed to be a knot invariant. Prove either useful approximation guarantees under a restricted move system or obstruction families showing why a proposed local system cannot achieve them.

Research question 12.9 (A certified diagram-to-grammar producer). Can the general diagram pipeline produce compact braid words and a proof of their closure equivalence, while preserving compression during simplification? The relevant measure is the sum of discovery time, proof size, and replay time. A small final grammar is not enough if an exponentially expanded intermediate object was needed to find it.

Research question 12.10 (Marked collars and geometric hierarchies). Can braid-encoded collars or parallelity regions inside a geometric hierarchy be replaced by compressed certified objects while retaining meridian, longitude, and attachment data? A successful interface would connect the present exact algebra to the geometric program. An unmarked closed-braid classification is insufficient for substitution into an arbitrary three-manifold boundary pattern.

Research question 12.11 (An end-to-end exceptional-parameter theorem). Find a natural nontrivial diagram class for which a certified polynomial-size transformation always yields $\kappa_{\text{exc}} = O(\log^2 n)$. A stronger universal statement would meet the general goal, but even a precise structural class beyond planted connected sums would be meaningful. The theorem must describe how the transformation is discovered, not merely assert the existence of a favorable representation.

Research question 12.12 (Fallback and budget integration). Wire the verified exceptional leaves into the maintained complete backend with a shared global resource ledger and preserved child certificates. Measure crossover thresholds for explicit versus compressed leaves and verify that a fallback timeout never changes an unresolved overall status into an affirmative one. This is an implementation milestone, not a new mathematical assumption in the hybrid bound.

Research question 12.13 (Independent nonplanted evaluation). Build a reproducible corpus from actual grammar-producing project workflows, rather than only powers and known conjugates. Keep explicit-input and compressed-input populations separate. Report unsuccessful decompositions, no-progress overhead, certificate size, replay cost, and whole-pipeline cost. An exact algebraic stress test and an operational knot benchmark answer different questions and should remain separately labeled.

13 Conclusion

The concrete achievement is a complete native compressed recognizer for three-braid knot closures, with controlled grammar growth and a replayable proof for every reported decision. Its connected-sum extension preserves this advantage at arbitrary strand count whenever all factors fall into the supported family. A matrix materialization lower bound explains why this gain is not obtained automatically from a compact product notation.

For the broader quasi-polynomial program, the useful new accounting rule is to charge the exponential backend only for *exceptional* factors. The three-braid-exempt minority bound permits huge compressed easy factors without placing their crossing or minority counts in that exponent. The missing general step is still a provably efficient producer of favorable decompositions. The article and artifacts make that remaining obligation explicit, while supplying a tested, exact component that such a producer can use.

A Reproduction and artifact map

From the extracted package directory:

```
python -m unittest discover -s tests -v
python experiments/run_audit.py
python experiments/benchmark.py
python experiments/generate_examples.py
python experiments/render_tables.py
sh paper/build.sh
```

The direct recognition and replay commands are:

```
python -m compressed_b3 recognize examples/sleeve_256.json
python -m compressed_b3 verify examples/sleeve_256.json \
  --certificate examples/sleeve_256.result.json
python -m compressed_b3 forest examples/singleton_forest_12.json
python -m compressed_b3 verify-forest examples/singleton_forest_12.json \
  --certificate examples/singleton_forest_12.result.json
```

The library modules are `grammar.py` for input semantics, `strings.py` for exact SLP operations, `engine.py` for quotient discovery, `verify.py` for replay, and `forest.py` for arbitrary-strand singleton projection. `experiments/oracles.py` supplies the local controls. No third-party manuscript PDFs or font files are included. The embedded bibliography permits offline compilation of this source.

B Claim ledger

Claim	Status and scope
Explicit three-braid classification	Prior project result, audited algebraically here; not claimed as a new discovery.
Polynomial compressed conjugacy	Established prior theory for the quotient; not a new general complexity theorem.
One-LCP reduction and polynomial grammar bound	Proved here for the stated native binary grammar model; implemented and locally tested.
Independent replay	Separate procedural verifier, no producer or LCP search; shared exact string kernel and imported topology remain trusted.
Compressed singleton forest	Implemented; complete when every leaf has at most three strands. Classical splitting topology is credited.

Claim	Status and scope
Hybrid complete recognition bound	Proved for composition with a complete exact exponential fallback. That fallback integration is not delivered as a completed production feature.
General quasi-polynomial recognition	Not established by this article. The missing decomposition/discovery theorem is stated explicitly.
Local performance measurements	Completed synthetic native-input comparisons, with replay and raw samples. Not a full production speedup claim.
Upstream regression and Rust	Not run. The optional real-checkout compatibility check is supplied, not certified as passed.

References

- [1] V. Reshetnikov, *ProveIt: Topology/UnknotRecognition*, repository snapshot 47002f64b9a97f64edc8e8b8f793b83983b719f9. In particular, `fast/fastunknot/braid.py`, `compressed_words.py`, and `fast/README.md`. Pinned repository directory. Inspected 8 October 2026.
- [2] ProveIt research report 19, *Optimal rank-two braid kernels and survivor growth*, 7 October 2026. `Topology/UnknotRecognition/reports/19/README.md`, at the same pinned repository reference.
- [3] ProveIt research report 34, *Certified braid kernels and linear Markov descent*, 8 October 2026. `Topology/UnknotRecognition/reports/34/README.md`, at the same pinned repository reference.
- [4] S. Schleimer, *Polynomial-time word problems*, Comment. Math. Helv. **83** (2008), 741–765. DOI: 10.4171/CMH/142. arXiv:math/0608563.
- [5] D. Holt, M. Lohrey, and S. Schleimer, *Compressed decision problems in hyperbolic groups*, Groups Geom. Dyn. **18** (2024), 1233–1273. DOI: 10.4171/GGD/809. arXiv:1808.06886.
- [6] D. Bennequin, *Entrelacements et équations de Pfaff*, Astérisque **107–108** (1983), 87–161. NUMDAM edition.
- [7] A. Conway, *Braid maps and twisted Alexander polynomials*, arXiv:1510.06678, version 2. Used for the closed-braid Alexander determinant formula.
- [8] C. A. Van Cott, *Relationships between braid length and the number of braid strands*, Algebr. Geom. Topol. **7** (2007), 181–196. DOI: 10.2140/agt.2007.7.181. arXiv:math/0605476.
- [9] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publ. Math. IHÉS **113** (2011), 97–208. DOI: 10.1007/s10240-010-0030-y. arXiv:1005.4346.
- [10] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. In particular Sections 9–10.
- [11] T. Kelomäki and D. Schütz, *On computational complexity of Khovanov homology*, arXiv:2601.02119v1, 5 January 2026.